

Verification Package

Secure Notes Application (SUD)

SE 300 – Software Engineering Practices
Spring 2026

Authors:

TJ Drape

Afonso Azevedo

Roman Zalewski

John Markham

Embry-Riddle Aeronautical University
Prescott, Arizona

April 23, 2026

Contents

Project Repository	2
1 Verification Philosophy	2
2 Test Cases and Test Procedures	3
2.1 Summarize Note	3
2.2 Display Note	6
2.3 Create Note	8
2.4 Change Note	11
2.5 Delete Note	13
2.6 Exit Program	16
2.7 Get Password	17
3 Code Reviews	19

Project Repository

The full source code for the Secure Notes Application is available at:

[Secure Notes GitHub Repository](#)

1 Verification Philosophy

The goal of the verification process for the Secure Notes Application (SUD) is to ensure that the system is correct and meets the requirements defined in the Software Requirements Document (SRD).

We performed both unit testing and integration testing. Unit testing was used to verify individual modules, while integration testing ensured that all modules work correctly together through the Menu controller.

We applied both black-box and white-box testing. Black-box testing focused on validating the system from the user's perspective using inputs and expected outputs based on the SRD. White-box testing focused on verifying the internal logic of the code, including conditions, data processing, and edge cases.

We also tested boundary values and edge cases such as empty inputs, large inputs, and invalid data types to ensure the system behaves reliably under different conditions.

In addition, we performed informal code reviews by comparing the implementation with the SDD to ensure consistency and correctness.

Overall, this approach allows us to confirm that the system works as expected and follows its design.

2 Test Cases and Test Procedures

2.1 Summarize Note

Field	Description
Test Case ID	TC-AI-01
Test Case Name	Basic Summarization Functionality
Trace Tag	SDD_TT_2.001 :: SDD_TT_2.003 :: SDD_HLD_SUM.001 :: [SRD::T_12]
Objective	Verify that the system generates a non-empty summary shorter than the input text for a normal multi-sentence note.
Approach	Black-box testing using a normal multi-sentence note as input. The output is checked for correctness and length.
Test Procedures	TP-AI-01-01: Launch the application and enter a valid password. TP-AI-01-02: Create or select a normal multi-sentence note. TP-AI-01-03: Select option 4 (Summarize Note) from the main menu. TP-AI-01-04: Choose the note and generate the summary. TP-AI-01-05: Observe the summary output.
Expected Result	The system returns a non-empty summary that is shorter than the original note and preserves key ideas from the input.
Actual Result	The system generated a valid summary. The summary was non-empty, shorter than the original note, and preserved the main ideas.
Test Status	Passed
Implementation Reference	<code>menu.py</code> , <code>ai_processor.py</code> , <code>summarizer.py</code>

Field	Description
Test Case ID	TC-AI-02
Test Case Name	Summarize Note with No Existing Notes
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_008 :: SDD_HLD_MENU_001 :: [SRD::T_12]
Objective	Verify that the system safely handles a request to summarize a note when no notes exist.
Approach	Black-box testing. The user selects summarize note when the system has no saved notes.
Test Procedures	TP-AI-02-01: Launch the application and enter a valid password. TP-AI-02-02: Ensure that no notes exist in the system. TP-AI-02-03: Select option 4 (Summarize Note) from the main menu. TP-AI-02-04: Observe the system response.
Expected Result	The system should notify the user that no notes are available and return to the main menu without crashing.
Actual Result	The system displayed the message “Error with file. Aborting and returning to Menu.” and returned to the menu without crashing.
Test Status	Passed
Implementation Reference	menu.py, db.py, ai_processor.py

Field	Description
Test Case ID	TC-AI-03
Test Case Name	Summarize Short Note
Trace Tag	SDD_TT_2_001 :: SDD_TT_2_003 :: SDD_HLD_SUM_001 :: [SRD::T_12]
Objective	Verify that the system correctly handles summarization of a short note with minimal content.
Approach	Black-box testing using a single-sentence note. The output is checked for correctness and system stability.
Test Procedures	TP-AI-03-01: Launch the application and enter a valid password. TP-AI-03-02: Create a note with the content “Hello my name is Afonso.” TP-AI-03-03: Select option 4 (Summarize Note) from the main menu. TP-AI-03-04: Choose the created note and generate the summary. TP-AI-03-05: Observe the summary output.
Expected Result	The system should return a valid summary or the original sentence if no reduction is possible, without crashing.
Actual Result	The system returned the original sentence “Hello, I like Machine Learning.” as the summary and did not crash.
Test Status	Passed
Implementation Reference	menu.py, ai_processor.py, summarizer.py

2.2 Display Note

Field	Description
Test Case ID	TC-DIS-01
Test Case Name	Basic Display Note Functionality
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_006 :: SDD_HLD_MENU_001 :: [SRD::T_14]
Objective	Verify that the system displays an existing note correctly.
Approach	Black-box testing using a saved note in the system.
Test Procedures	TP-DIS-01-01: Launch the application and enter a valid password. TP-DIS-01-02: Ensure that at least one note exists. TP-DIS-01-03: Select option 2 (Display Note). TP-DIS-01-04: Choose an existing note. TP-DIS-01-05: Observe the displayed content.
Expected Result	The selected note is displayed correctly and matches the saved content.
Actual Result	The system displayed the selected note correctly. The content matched the saved note and was readable.
Test Status	Passed
Implementation Reference	menu.py, db.py, security.py

Field	Description
Test Case ID	TC-DIS-02
Test Case Name	Display Note with No Existing Notes
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_006 :: SDD_HLD_MENU_001 :: [SRD::T_14]
Objective	Verify that the system safely handles a user request to display a note when no notes currently exist in storage.
Approach	Black-box testing. The user selects the display note option when the system has no saved notes. The response is evaluated based on whether the program handles the condition safely and returns useful feedback to the user.
Test Procedures	TP-DIS-02-01: Launch the application and enter a valid password. TP-DIS-02-02: Ensure that no notes currently exist in the system. TP-DIS-02-03: Select option 2 (Display Note) from the main menu. TP-DIS-02-04: Observe the system response.
Expected Result	The system should notify the user that no notes are available to display and return to the main menu without crashing.
Actual Result	The system displays the error message: "Error with file. Aborting and returning to Menu." It safely returns the user to the menu without crashing.
Test Status	Passed
Implementation Reference	menu.py, db.py

2.3 Create Note

Field	Description
Test Case ID	TC-CREATE-01
Test Case Name	Valid Create Note Functionality
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_002, SDD_HLD_MENU_003] LLD : [SDD_TT_3_001, SDD_TT_3_002, SDD_TT_3_005, SDD_TT_3_010, SDD_TT_3_012] SRD : [SRD_T_02, SRD_T_04, SRD_T_05, SRD_T_11, SRD_T_19, SRD_T_21]
Objective	Verify that the system allows a user to create, encrypt, and save a non-null note.
Approach	Black-box testing. The system follows the process for a user to create a new note. It tests from entering a valid password, selecting the proper action number, adding data to the note, to then encrypting the note and saving it to the database.
Test Procedures	Note: The following will be run after the TSC. TP-CREATE-01-01: When prompted for a password, enter a valid password of at least 8 characters, then press Enter. TP-CREATE-01-02: When prompted to select an action, press 1 and Enter. TP-CREATE-01-03: When prompted with "'filename':", then write at least 1 character to the terminal, and press Enter.
Expected Result	The system saves an encrypted version of the file to the DB (notes.db).

Procedure to Observe Result	To ensure that the system correctly saves a file to the DB with the same name, run the following commands: sqlite3 path_to_your_database (Note: This would be .schema notes.db) SELECT * FROM notes WHERE note_name = generated_file_name;
Actual Result	The system correctly prompts the user to input data for a note, it validly encrypts this data, and then correctly saves the file to the DB.
Test Status	Passed
Implementation Reference	menu.py, security_manager.py, db.py, notes.db

Field	Description
Test Case ID	TC-CREATE-02
Test Case Name	Disallow Empty Note Save Functionality
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_002, SDD_HLD_MENU_003] LLD : [SDD_TT_3_001, SDD_TT_3_002, SDD_TT_3_005, SDD_TT_3_010, SDD_TT_3_012] SRD : [SRD_T_06, SRD_T_21, SRD_T_44, SRD_T_45]
Objective	Verify that the system allows a user to create, encrypt, and save a non-null note.
Approach	Black-box testing. The system follows the process for a user trying to save an empty note to the DB. The user will enter a valid password, select the Create Note action, and then write no data to the file.

Test Procedures	Note: The following will be run after the TSC. TP-CREATE-02-01: When prompted for a password, enter a valid password of at least 8 characters, then press Enter. TP-CREATE-02-02: When prompted to select an action, press 1 and Enter. TP-CREATE-02-03: When prompted with "'filename':", press Enter.
Expected Result	The system prints an error message to the user, aborts the note creation process, and returns the user to the menu.
Procedure to Observe Result	To ensure that the system did not save the file, run the following commands: <pre>sqlite3 path_to_your_database (Note: This would be .schema notes.db)</pre> <pre>SELECT * FROM notes WHERE note_name = generated_file_name;</pre> (This query should yield no results)
Actual Result	The system properly handled the null note, displayed an error message to the user, and gracefully returned them to the menu.
Test Status	Passed
Implementation Reference	menu.py, notes.db

2.4 Change Note

Field	Description
Test Case ID	TC-CHG-01
Test Case Name	Basic Change Note Functionality
Trace Tag	SDD_TT_3_004 :: SDD_HLD_MENU_001 :: [SRD::T_10]
Objective	Verify that the system allows the user to modify an existing note by appending new content.
Approach	Black-box testing using an existing note. The updated content is verified after modification.
Test Procedures	TP-CHG-01-01: Launch the application and enter a valid password. TP-CHG-01-02: Create a note with the content “Hello, I like Machine Learning.” TP-CHG-01-03: Select option 3 (Change Note) from the main menu. TP-CHG-01-04: Choose the created note. TP-CHG-01-05: Append new content “I also like to study.” TP-CHG-01-06: Save the changes. TP-CHG-01-07: Display the note to verify the update.
Expected Result	The system appends the new content to the existing note without overwriting previous data. The updated note should contain both the original and appended text.
Actual Result	The system appended the new content successfully. The final note displayed was “Hello, I like Machine Learning. I also like to study.”
Test Status	Passed
Implementation Reference	menu.py, db.py

Field	Description
Test Case ID	TC-CHG-02
Test Case Name	Change Note with No Existing Notes
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_007 :: SDD_HLD_MENU_001 :: [SRD::T_13]
Objective	Verify that the system safely handles a request to change a note when no notes exist.
Approach	Black-box testing. The user selects change note when the system has no saved notes.
Test Procedures	TP-CHG-02-01: Launch the application and enter a valid password. TP-CHG-02-02: Ensure that no notes exist in the system. TP-CHG-02-03: Select option 3 (Change Note) from the main menu. TP-CHG-02-04: Observe the system response.
Expected Result	The system should notify the user that no notes are available to change and return to the main menu without crashing.
Actual Result	The system displayed the message “Error with file. Aborting and returning to Menu.” and returned to the menu without crashing.
Test Status	Passed
Implementation Reference	menu.py, db.py

2.5 Delete Note

Field	Description
Test Case ID	TC-DEL-01
Test Case Name	Delete Note with No Existing Notes
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_009 :: SDD_HLD_MENU_001 :: [SRD::T_14]
Objective	Verify that the system safely handles a request to delete a note when no notes exist.
Approach	Black-box testing. The user selects delete note when the system has no saved notes.
Test Procedures	TP-DEL-01-01: Launch the application and enter a valid password. TP-DEL-01-02: Ensure that no notes exist in the system. TP-DEL-01-03: Select option 5 (Delete Note) from the main menu. TP-DEL-01-04: Observe the system response.
Expected Result	The system should notify the user that no notes are available to delete and return to the main menu without crashing.
Actual Result	The system displayed the message “Error with file. Aborting and returning to Menu.” and returned to the menu without crashing.
Test Status	Passed
Implementation Reference	menu.py, db.py

Field	Description
Test Case ID	TC-DEL-02
Test Case Name	Basic Delete Note Functionality
Trace Tag	SDD_TT_3_003 :: SDD_TT_3_004 :: SDD_TT_3_009 :: SDD_HLD_MENU_001 :: [SRD::T_23]
Objective	Verify that the system allows the user to delete an existing note after confirmation.
Approach	Black-box testing using an existing saved note. The user selects the note, confirms deletion, and the system response is observed.
Test Procedures	TP-DEL-02-01: Launch the application and enter a valid password. TP-DEL-02-02: Create a note with sample content. TP-DEL-02-03: Select option 5 (Delete Note) from the main menu. TP-DEL-02-04: Choose the created note. TP-DEL-02-05: Confirm deletion by entering “y”. TP-DEL-02-06: Observe the system response and return to the main menu.
Expected Result	The system should request confirmation, delete the selected note after confirmation, and return to the main menu without crashing.
Actual Result	The system displayed the selected note, requested confirmation, accepted the input “y”, deleted the note, and returned to the main menu without crashing.
Test Status	Passed
Implementation Reference	menu.py, db.py

Field	Description
Test Case ID	TC-DEL-03
Test Case Name	Invalid Note Name Delete Functionality
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_002, SDD_HLD_MENU_003] LLD : [SDD_TT_3.003, SDD_TT_3.004, SDD_TT_3.009] SRD : [SRD_T_44, SRD_T_45]
Objective	Verify that the system disallows a user from deleting a note that does not exist.
Approach	Black-box testing using a filename that does not exist in the DB. The user selects the note and the system response is observed.
Test Procedures	TP-DEL-03-01: Enter a valid password. TP-DEL-03-02: Select option 5 (Delete Note) from the main menu. TP-DEL-03-03: Choose a filename not shown in the displayed file list. TP-DEL-03-04: Observe the system response and return to the main menu.
Expected Result	The system should display an error message to the user, abort the delete action, and return to the menu.
Actual Result	The system displayed an error message about correct filename, displayed a separate message about error with file, aborted the delete action, and returned the user to the menu.
Test Status	Passed
Implementation Reference	menu.py, db.py

2.6 Exit Program

Field	Description
Test Case ID	TC-EXIT-01
Test Case Name	Exit Program from Menu
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_003] LLD : [SDD_TT_3.010, SDD_TT_3.012] SRD : [SRD_T.04, SRD_T.22]
Objective	Verify that the system correctly allows a user to exit the program from the menu.
Approach	Black-box testing. The user selects the "Exit Program" action from the menu.
Test Procedures	TP-EXIT-01-01: Launch the application and enter a password. TP-EXIT-01-02: When prompted to select an action, press 6 and Enter. (Exit Program) TP-EXIT-01-03: Observe the system response.
Expected Result	The system should display the exit message, then display the farewell message and exit the program.
Actual Result	The system displayed the exit message, then displayed the farewell message before exiting the program.
Test Status	Passed
Implementation Reference	menu.py

2.7 Get Password

Field	Description
Test Case ID	TC-PASSWORD-01
Test Case Name	Accepts a Valid 8-character Password
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_003] LLD : [SDD_TT_03.011] SRD : [SRD_T_28, SRD_T_31]
Objective	Verify that the system correctly allows a user to set the password if it meets the character requirement.
Approach	Black-box testing. The user enters an 8-character password when prompted.
Test Procedures	TP-PASSWORD-01-01: Launch the application. TP-PASSWORD-01-02: When prompted, enter an 8-character password. TP-PASSWORD-01-03: Observe the system response.
Expected Result	The system should accept the password and then display the menu.
Actual Result	The system accepted the password, and then displayed the menu to the user.
Test Status	Passed
Implementation Reference	menu.py

Field	Description
Test Case ID	TC-PASSWORD-02
Test Case Name	Disallows Invalid Password
Trace Tag	HLD : [SDD_HLD_MENU_001, SDD_HLD_MENU_003] LLD : [SDD_TT_03_011] SRD : [SRD_T_28, SRD_T_31, SRD_T_44, SRD_T_45]
Objective	Verify that the system correctly disallows invalid passwords, and re-prompts the user for a valid password. Then, it allows a valid password.
Approach	Black-box testing. The user enters an invalid password, and we observe how the system responds.
Test Procedures	TP-PASSWORD-02-01: Launch the application. TP-PASSWORD-02-02: When prompted, enter an invalid password. (length less than 8 characters) TP-PASSWORD-02-03: Observe the system response. TP-PASSWORD-02-04: When prompted, enter a valid 8-character password. TP-PASSWORD-02-05: Observe the system response.
Expected Result	The system disallow the invalid password, re-prompt the user for a password, accept the valid password, and then display the menu to the user.
Actual Result	The system recursively prompts the user for a password until they enter a valid one, and then displays the menu.
Test Status	Passed
Implementation Reference	menu.py, security_manager.py

3 Code Reviews

Our test cases were essential to ensure our code runs smoothly. By reviewing each module before integration, we caught logical errors early in the process and maintained code sustainability across the project.

Code Reviews:

Throughout the development process, we relied heavily on virtual, asynchronous reviews. When we completed part of the project, we submitted the code with a detailed commit message to our remote GitHub repository. Then we tested how and where this part integrated into the application. During this process, if we encountered any issues, we noted them, and then updated the affected functions appropriately. After this, we repeat this process, highlighting our iterative approach to development.

Analysis Procedures:

Once we had a working system, we reviewed and refactored our code based on our SRD requirements. If we noticed a discrepancy between our code and the SRD, we deferred to the requirements. However, we also changed how some implemented requirements worked to improve design and user experience. For example, prior to the development period, we noted that the user should be able to exit the program with (Ctrl+C). However, as the SUD took shape, this exiting method seemed out of place, as it would show multiple interruption error messages. So, we changed the exit command so that the user has to enter "6" at the menu. This provided the program a cleaner exit method, showing how we analyzed the program to give the user a better experience.